ARTIFICIAL NEURAL NETWORKS (ANN)
LAB 05

Lab Demonstrator: Shakeela Shaheen

LAB 04: IMPLEMENTING DROPOUT, BATCH NORMALIZATION, AND OPTIMIZERS NEURAL NETWORKS

LEARNING OBJECTIVES:

By the end of this lab, students will:

- Build deep neural networks using TensorFlow.
- Understand and implement **Dropout**.
- Implement **Batch Normalization**.
- Compare optimizers (SGD vs Adam).
- Diagnose overfitting using validation curves.
- Analyze training stability

OVERVIEW:

In Lab 03, you built a deep MLP. But now we ask a more serious question: Why do deep networks sometimes:

- Overfit?
- Train slowly?
- Become unstable?

This week is about **making networks stable and generalizable.** You will experiment, observe, and conclude.

1 – Why Do Neural Networks Overfit?

A neural network with many parameters has **high capacity**.

High capacity means:

- It can memorize training data.
- It may fail to generalize.

Overfitting happens when:

Training Loss ↓ Validation Loss ↑

This means:

- Model learned noise.
- Model memorized patterns.
- Model lost generalization.

2 – Dropout:

Dropout is a regularization technique proposed in 2014.

During training:

- Randomly deactivate (drop) neurons.
- With probability p.

Mathematically:

If neuron output is:

$$a_i$$

After dropout:

$$\tilde{a}_i = \begin{cases} 0 & \text{with probability } p \\ \dfrac{a_i}{1-p} & \text{otherwise} \end{cases}$$

Why Does Dropout Work?

Because:

- Neurons cannot rely on specific other neurons.
- Forces network to learn redundant representations.
- Prevents co-adaptation.

Imagine a football team. If one player always carries the team, removing him makes team collapse. Dropout forces:

Every player must learn to contribute.

```
model = Sequential([
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
```

3 – Batch Normalization:

During training:

- Layer1 weights update
- Therefore Layer1 outputs change
- Therefore input distribution to Layer2 changes
- Therefore Layer2 must readjust
- Then Layer3 must readjust

This happens **every iteration**.

So deeper layers are chasing a moving target.

First understand "covariate shift". Covariate = input distribution.

If:

$$P_{train}(x) \neq P_{test}(x)$$

that is covariate shift.

Now internal covariate shift means:

The input distribution of a hidden layer keeps changing during training.

Mathematically:

Let:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

But since:

$$a^{(l-1)}$$

depends on weights of previous layer, and those weights change each iteration,

then the distribution:

$$P(z^{(l)})$$

keeps shifting.

That makes optimization unstable.

Why Is That a Problem?

Because neural networks train using gradient descent.

Gradient descent assumes:

The function landscape is somewhat stable.

But if the input distribution keeps shifting:

- Gradients fluctuate
- Learning becomes slower
- Small learning rates are required
- Convergence slows down

The Core Idea of Batch Normalization

BN says:

Let's force activations of each layer to have mean ≈ 0 and variance ≈ 1.

For each mini-batch.

That stabilizes distribution.

Full Mathematical Process of BN

Assume mini-batch size = m.

For one neuron output values:

$$x_1, x_2, \ldots, x_m$$

Step 1 — Compute Batch Mean

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

Why?

We want to center the data.

Step 2 — Compute Batch Variance

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

Why?

We want to measure spread.

Step 3 — Normalize

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Now:

- Mean ≈ 0
- Variance ≈ 1

ε is added for numerical stability.

Important Question

If we normalize everything to mean 0 and variance 1, Are we restricting the network too much?

YES — unless we fix it.

So BN introduces learnable parameters:

Step 4 — Scale and Shift:

$$y_i = \gamma\hat{x}_i + \beta$$

Where:

- γ (gamma) is learnable scale
- β (beta) is learnable shift

This allows network to recover any distribution if needed.

So BN does:

Normalize → then allow flexibility.

Why BN Reduces Overfitting (Somewhat)

Because:

- Each mini-batch has slightly different mean/variance
- So normalization is slightly noisy

That noise acts like regularization. Not as strong as Dropout, but still helpful.

Imagine teaching a class.

Without BN:

Every lecture:

- Students come with completely different background levels.
- You must adjust constantly.

With BN:

Before lecture:

- Everyone is normalized to same knowledge baseline.

Now teaching becomes stable and efficient.

```
model = Sequential([
    Dense(64),
    BatchNormalization(),
    Activation('relu'),
    Dense(32),
    BatchNormalization(),
    Activation('relu'),
    Dense(1, activation='sigmoid')
])
```

4 – Optimizers

Gradient Descent Recap

Basic update rule:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Where:

- η = learning rate

SGD (Stochastic Gradient Descent)

Uses gradient from small batches.

Pros:

- Simple
- Works well

Cons:

- Can oscillate
- Sensitive to learning rate

Adam Optimizer

Adam = Adaptive Moment Estimation.

It keeps:

- Exponential moving average of gradients
- Exponential moving average of squared gradients

Update rule:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Correct bias:

$$\hat{m_t} = \frac{m_t}{1 - \beta_1^t}$$
$$\hat{v_t} = \frac{v_t}{1 - \beta_2^t}$$

Final update:

$$w = w - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Why Adam Is Powerful

- Adaptive learning rate
- Faster convergence
- Less tuning needed

LAB TASKS

You can use:

- Breast Cancer dataset (recommended)
  OR
- Heart Disease dataset
  OR
- MNIST

Task 1 – Baseline Model

Build deep network WITHOUT:

- Dropout
- BatchNorm

Train 50 epochs.

Plot:

- Training loss
- Validation loss
- Accuracy

Record results.

Task 2 – Add Dropout

Modify model.

Train again.

Compare:

- Overfitting behavior
- Accuracy

Task 3 – Add BatchNorm

Train and compare:

- Convergence speed
- Stability
- Final performance

Task 4 – Combine Dropout + BatchNorm

Train and analyze.

Task 5 – Optimizer Comparison

Train same architecture with:

- SGD
- Adam

Plot loss curves on same graph.

Compare:

- Speed
- Smoothness
- Final performance

Task 6 – Learning Rate Sensitivity

Try:

- lr = 0.0001
- lr = 0.01
- lr = 0.5

Observe instability or divergence.

Final Submission Requirements

Students must submit:

1. Complete notebook
2. All loss plots
3. Comparison table
4. Brief analysis (max 2-4 paras at end)